HUMLAI POS · ระบบขายหน้าร้านหลายสาขา

คู่มือติดตั้งระบบใหม่

ตั้งแต่เครื่องเปล่า จนเปิดขายได้ และการเพิ่มสาขาใหม่

ฉบับวันที่ 23 กันยายน 2026 · ตรงกับโค้ดรุ่น `2026-09-23-stock`
ต้นฉบับอยู่ที่ `docs/install-guide/INSTALL-GUIDE.html` ใน GitHub (Narai-IT/HumLai-pos)

1. ภาพรวมระบบและสิ่งที่ต้องเตรียม
2. เตรียมเครื่องเซิร์ฟเวอร์และ SQL Server
3. ติดตั้ง API Server
4. เปิดทางเข้าด้วย Cloudflare Tunnel
5. ติดตั้งหน้าเว็บบน Vercel
6. ตั้งค่าร้านในหลังบ้าน
7. ติดตั้ง Print Server ในร้าน
8. เปิดล็อก API
9. เปิดสาขาใหม่
10. การอัปเดตระบบ
11. ความปลอดภัย
12. แก้ปัญหาที่เจอบ่อย

1. ภาพรวมระบบและสิ่งที่ต้องเตรียม

ระบบมี 3 ส่วน: **หน้าเว็บ** อยู่บน Vercel, **API + ฐานข้อมูล** อยู่บนเครื่องเซิร์ฟเวอร์เครื่องเดียว (ใช้ร่วมกันทุกสาขา) และ **Print Server** อยู่ในร้านแต่ละสาขา สาขาละหนึ่งเครื่อง



สิ่งที่ต้องมี

รายการ	รายละเอียด
เครื่องเซิร์ฟเวอร์ 1 เครื่อง	Windows 10/11 หรือ Windows Server เปิดทิ้งไว้ตลอด ใช้ติดตั้ง SQL Server + API Server + cloudflared
เครื่อง Print Server สาขาละ 1 เครื่อง	Windows ในวงแลนเดียวกับปริ้นเตอร์ เปิดตลอดเวลาที่ร้านเปิด (ใช้เครื่องแคชเชียร์ได้)
ปริ้นเตอร์ความร้อน ESC/POS	ต่อสาย LAN ตั้ง IP คงที่ รับงานพอร์ต 9100
บัญชีและบริการ	GitHub (เข้า repo `Narai-IT/HumLai-pos` ได้), Vercel, Cloudflare + โดเมนที่ย้าย DNS มาที่ Cloudflare แล้ว, SlipOK (ถ้ารับโอนผ่าน QR)
โปรแกรม	SQL Server (Express ก็ได้) + SQL Server Management Studio, Node.js **20 ขึ้นไป** (LTS), Git for Windows

พอร์ตที่ใช้

พอร์ต	ของ	เปิดจากที่ไหน
1433	SQL Server	ในเครื่องเซิร์ฟเวอร์เท่านั้น **ห้ามเปิดออกอินเทอร์เน็ต**
8080	API Server	ในเครื่องเซิร์ฟเวอร์ (Cloudflare Tunnel ต่อเข้ามาเอง)
3001	Print Server	ในเครื่อง/วงแลนของร้าน
9100	ปริ้นเตอร์	ในวงแลนของร้าน

ลำดับการติดตั้ง
ทำตามบทที่ 2 → 8 ตามลำดับ แต่ละบทมีจุดทดสอบ ผ่านแล้วค่อยไปบทถัดไป ถ้าติดให้ดูบทที่ 12

2. เตรียมเครื่องเซิร์ฟเวอร์และ SQL Server

2.1 ติดตั้งโปรแกรม

1. **SQL Server** (Express ใช้ได้) + **SQL Server Management Studio (SSMS)**
2. **Node.js LTS** จาก `https://nodejs.org` (ต้องรุ่น 20 ขึ้นไป) — ติดตั้งแบบค่าเริ่มต้น
3. **Git for Windows** จาก `https://git-scm.com/download/win` — ติดตั้งแบบค่าเริ่มต้น (เปิด Git Credential Manager ไว้)

เช็กใน PowerShell (ปิดแล้วเปิดใหม่หลังติดตั้ง): `node -v` ต้องได้ v20 ขึ้นไป และ `git --version` ต้องขึ้นเลขรุ่น

2.2 ตั้งค่า SQL Server

1. เปิด **SQL Server Configuration Manager** → SQL Server Network Configuration → Protocols → เปิด **TCP/IP** → รีสตาร์ตเซอร์วิส SQL Server
2. ใน SSMS คลิกขวาที่เซิร์ฟเวอร์ → Properties → Security → เลือก **SQL Server and Windows Authentication mode** → รีสตาร์ตเซอร์วิส

2.3 สร้างฐานข้อมูลและผู้ใช้ `pos_app`

ใน SSMS เปิด New Query แล้วรัน (เปลี่ยนรหัสผ่านเป็นของจริง ยาวและเดายาก):

```
CREATE DATABASE HumLaiPOS;
GO
USE HumLaiPOS;
GO
CREATE LOGIN pos_app WITH PASSWORD = 'ตั้งรหัสผ่านที่เดายาก';
CREATE USER  pos_app FOR LOGIN pos_app;
ALTER ROLE db_owner ADD MEMBER pos_app;   -- ต้องสร้าง/แก้ตารางได้ตอนรัน sql:init
GO
```

จดรหัสผ่านไว้ในที่ปลอดภัย
ใช้ใส่ในไฟล์ `.env` ในบทที่ 3 ห้ามส่งในแชท ห้ามใส่ในไฟล์ที่ขึ้น GitHub

3. ติดตั้ง API Server

3.1 สร้าง token อ่านอย่างเดียวของ GitHub

repo เป็นแบบส่วนตัว เครื่องเซิร์ฟเวอร์ต้องมีสิทธิ์อ่าน ใช้ token ที่อ่านได้เฉพาะ repo นี้ แทนการล็อกอินบัญชีของคน

1. GitHub → รูปโปรไฟล์ → Settings → Developer settings → Personal access tokens → **Fine-grained tokens** → Generate new token
2. Resource owner: `Narai-IT` · Repository access: เฉพาะ `HumLai-pos` · Permissions → Contents: **Read-only** · Expiration: 1 ปี (จดวันหมดอายุ)
3. คัดลอก token เก็บไว้ (เห็นได้ครั้งเดียว)

3.2 ดึงโค้ดลงเครื่อง

เปิด **PowerShell แบบ Run as administrator**:

```
cd D:\
git config --global credential.useHttpPath true
git clone https://github.com/Narai-IT/HumLai-pos.git humlai-pos
cd D:\humlai-pos
git config --global --add safe.directory D:/humlai-pos
```

ตอน clone จะมีหน้าต่างให้ล็อกอิน → เลือก **Token** แล้ววาง token จากข้อ 3.1 (Windows จำไว้ ครั้งหน้าไม่ถาม)

3.3 สร้างไฟล์ `.env`

```
copy .env.example .env
notepad .env
```

แก้ค่าเหล่านี้แล้วบันทึก:

```
SQL_SERVER=localhost
SQL_PORT=1433
SQL_DATABASE=HumLaiPOS
SQL_USER=pos_app
SQL_PASSWORD="รหัสผ่านจากข้อ 2.3"
SQL_ENCRYPT=false
SQL_TRUST_CERT=true
API_PORT=8080
```

รหัสผ่านต้องมีเครื่องหมายคำพูดครอบ
ถ้ารหัสมีตัว `#` แล้วไม่ครอบด้วย `"..."` ค่าจะถูกตัดตั้งแต่ตัว `#` แล้วขึ้น `Login failed for user 'pos_app'`

ถ้า SQL Server ติดตั้งเป็นอินสแตนซ์ชื่อ (เช่น `SQLEXPRESS`) ใช้ `SQL_SERVER=localhost\SQLEXPRESS` และลบบรรทัด `SQL_PORT`

3.4 ติดตั้งส่วนประกอบและสร้างตาราง

```
npm install
npm run sql:init
```

ต้องจบด้วย `✅ สร้าง/ตรวจสอบโครงสร้างครบ ... ชุดคำสั่ง` — รันซ้ำได้ ไม่มีข้อมูลหาย

3.5 ทดสอบ แล้วตั้งให้เปิดเองทุกครั้งที่เปิดเครื่อง

1. ดับเบิลคลิก `start-api.bat` แล้วเปิดเบราว์เซอร์ในเครื่องนั้น: `http://localhost:8080/api/pos?action=ping`
2. ต้องได้ `"db":"connected"` — ถ้าไม่ได้ ข้อความจะบอกสาเหตุ (ต่อไม่ติด / รหัสผิด / ยังไม่มีฐานข้อมูล)
3. ปิดหน้าต่าง `start-api.bat` แล้ว **คลิกขวา** `install-api-autostart.bat` → Run as administrator

ผ่านแล้วเมื่อ
รีสตาร์ตเครื่องแล้วเปิด `http://localhost:8080/api/pos?action=ping` ยังได้ `"db":"connected"` (API ทำงานเองโดยไม่ต้องมีคนล็อกอิน)

3.6 (ถ้ามี) ย้ายข้อมูลเดิมจาก Google Sheet

ใส่ `GAS_EXPORT_URL` ใน `.env` แล้วรัน `npm run sql:migrate -- --write` — รายละเอียดใน `docs/SQL-MIGRATION.md` ร้านที่เริ่มใหม่ข้ามข้อนี้ได้

4. เปิดทางเข้าด้วย Cloudflare Tunnel

ให้หน้าเว็บบน Vercel เรียก API ได้ โดยไม่ต้องเปิดพอร์ตขาเข้าที่เครื่องเซิร์ฟเวอร์ — เครื่องเป็นฝ่ายต่อออกไปหา Cloudflare เอง IP เปลี่ยนก็ไม่พัง

1. เข้า `https://one.dash.cloudflare.com` → **Networks → Tunnels → Create a tunnel** → เลือก **Cloudflared** → ตั้งชื่อ เช่น `humlai-pos`
2. เลือก **Windows 64-bit** → คัดลอกคำสั่ง `cloudflared.exe service install eyJ...` ไปรันใน PowerShell แบบ Run as administrator บนเครื่องเซิร์ฟเวอร์
   ถ้าไม่รู้จักคำสั่ง cloudflared ให้ติดตั้งจาก `https://github.com/cloudflare/cloudflared/releases/latest` (ไฟล์ .msi) ก่อน
3. รอสถานะเป็น **HEALTHY** → เพิ่ม Public Hostname:

Subdomain	Domain	Type	URL
`pos-api`	โดเมนของร้าน	HTTP	`localhost:8080`

4. ทดสอบจาก **มือถือที่ใช้เน็ตมือถือ** (ไม่ใช่ Wi-Fi ร้าน): `https://pos-api.<โดเมน>/api/pos?action=ping` ต้องได้ `"db":"connected"`
5. เพิ่ม 2 บรรทัดนี้ใน `D:\humlai-pos\.env` แล้วรัน `update-api.bat` (Run as administrator) เพื่อรีสตาร์ต API:

```
IMAGE_BASE_URL=https://pos-api.<โดเมน>
API_ALLOW_ORIGIN=https://hum-lai-pos.vercel.app
```

`IMAGE_BASE_URL` ทำให้รูปเมนูที่อัปโหลดเปิดได้ · `API_ALLOW_ORIGIN` จำกัดให้เฉพาะหน้าเว็บของร้านเรียก API จากเบราว์เซอร์ได้ (หลายโดเมนคั่นด้วยจุลภาค ไม่มี / ปิดท้าย)

มี Tunnel ของระบบอื่นอยู่แล้ว?
ไม่ต้องสร้างใหม่ — เปิด Tunnel เดิมแล้วเพิ่ม Public Hostname ตามข้อ 3 ได้เลย (ดู `docs/CLOUDFLARE-TUNNEL.md`)

5. ติดตั้งหน้าเว็บบน Vercel

1. Vercel → **Add New → Project** → Import repo `Narai-IT/HumLai-pos` (Framework: Vite — ใช้ค่าที่ Vercel ตรวจเจอ)
2. **Settings → Environment Variables** ตั้ง:

ชื่อ	ค่า
`VITE_API_URL`	`https://pos-api.<โดเมน>/api/pos` (ไม่มี / ปิดท้าย)
`SLIPOK_API_KEY`	คีย์จากหน้า SlipOK (ถ้ารับโอนผ่าน QR ที่ตรวจสลิปอัตโนมัติ)
`SLIPOK_BRANCH_ID`	Branch ID จาก SlipOK

   ไม่ต้องตั้ง `SQL_*` ที่ Vercel — หน้าเว็บคุยกับ API ผ่าน Tunnel ไม่ได้ต่อฐานข้อมูลเอง

3. กด **Deploy** (ทุกครั้งที่แก้ Environment Variables ต้อง **Redeploy** ถึงจะมีผล)
4. เปิดหน้าเว็บ → กด F12 → แท็บ Network → ต้องเห็นคำขอไปที่ `pos-api.<โดเมน>`

ผ่านแล้วเมื่อ
หน้าเว็บเปิดได้ ไม่มีแถบแดง — ระบบใหม่ยังไม่มีพนักงาน ให้กด **หน้าพนักงาน → เข้าเพื่อตั้งค่าพนักงานครั้งแรก** แล้วไปบทที่ 6

6. ตั้งค่าร้านในหลังบ้าน

ทำจากเครื่องไหนก็ได้ที่เปิดหน้าเว็บ ตามลำดับนี้ (ลำดับสำคัญ: สาขาต้องมาก่อนพนักงาน/โต๊ะ/ปริ้นเตอร์)

6.1 สาขา — หลังบ้าน → สาขา

1. **เพิ่มสาขา**: รหัสสาขา (เช่น `hl01` — ภาษาอังกฤษ/ตัวเลข **เปลี่ยนภายหลังไม่ได้**), ชื่อสาขา, ตัวนำหน้าเลขบิล (เช่น `HL01` → บิล `HL01-#001`)
2. กรอก ที่อยู่ เบอร์โทร เลขประจำตัวผู้เสียภาษี ข้อความท้ายใบเสร็จ — ใช้พิมพ์หัวใบเสร็จของสาขานั้น
3. กด **บันทึกทั้งหมด**

> ⭐ **สาขาหลัก**
> สาขาแรกที่เปิดใช้งานในรายการ = สาขาหลัก ข้อมูลที่ไม่ระบุสาขา (เครื่องที่ยังไม่ผูกสาขา, QR โต๊ะแบบเก่า) จะลงที่นี่ และแอดมินที่อยู่สาขาหลักนับเป็นแอดมินสำนักงานใหญ่

6.2 พนักงาน — หลังบ้าน → พนักงาน

ระดับสิทธิ์	ใช้กับ	สาขาที่ควรเลือก
👑 แอดมิน	เจ้าของ / ผู้จัดการ	🌐 ทุกสาขา = สำนักงานใหญ่ · สาขาเดียว = ผู้จัดการสาขานั้น
💳 แคชเชียร์	เก็บเงิน รับของเข้าสต็อก จัดผังโต๊ะ	สาขาที่ทำงาน
พนักงานทั่วไป	รับออเดอร์	สาขาที่ทำงาน

ตั้งรหัสผ่านให้ทุกคน และต้องมี **แอดมินที่ตั้งรหัสไว้อย่างน้อย 1 คน** ถึงจะล็อกอินเข้าหน้าพนักงานได้ และก่อนเปิดล็อก API ในบทที่ 8 · พนักงาน "ทุกสาขา" ล็อกอินแล้วจะมีหน้าให้เลือกสาขาที่ทำงานวันนั้น

6.3 ตั้งค่าร้าน — หลังบ้าน → ตั้งค่าร้าน

เซอร์วิสชาร์จ, VAT, QR รับเงิน (ตั้งแยกรายสาขาได้), หมายเหตุถึงครัว → กดบันทึก · สวิตช์ "ล็อก API" ให้เปิดในบทที่ 8

6.4 เมนู

1. **หมวดหมู่** → สร้างหมวด (อาหาร, เครื่องดื่ม, ...)
2. **จัดการเมนู** → เพิ่มเมนู ราคา รูป และเลือกปริ้นเตอร์ของเมนู (ตั้งหลังบทที่ 7) — นี่คือ **เมนูกลาง** ใช้ทุกสาขา
3. **เมนูรายสาขา** (ถ้าสาขาต่างกัน) → เลือกสาขา → ปิดไม่ขาย / ใส่ราคาเฉพาะสาขา / เลือกปริ้นเตอร์ของสาขา — ช่องว่าง = ตามเมนูกลาง
4. **BOM / สูตรอาหาร** และ **สต็อกวัตถุดิบ** (ถ้าใช้) — วัตถุดิบและสูตรใช้ร่วมกันทุกสาขา แต่ยอดคงเหลือแยกสาขา

6.5 โต๊ะและ QR — หลังบ้าน → จัดการโต๊ะ

1. ต้องเห็นบรรทัด 🏠 **ผังโต๊ะของสาขา:** ... ถูกสาขา (ผังโต๊ะแยกสาขา)
2. เพิ่ม/แก้โต๊ะ และประเภทราคา (ทานที่ร้าน / กลับบ้าน / เดลิเวอรี่)
3. กดปุ่ม QR ของแต่ละโต๊ะ → พิมพ์ติดโต๊ะ (ลิงก์มีรหัสสาขา ออเดอร์ที่ลูกค้าสั่งเองเข้าสาขาถูกต้อง)

7. ติดตั้ง Print Server ในร้าน

ทำที่เครื่อง Print Server ของสาขานั้น (อยู่วงแลนเดียวกับปริ้นเตอร์)

7.1 ติดตั้งโปรแกรม

วิธีง่ายสุด — ดาวน์โหลดจากหน้าเว็บ (ไม่ต้องใช้ Git)
หลังบ้าน → ตั้งค่าเครื่องพิมพ์ → **ดาวน์โหลด Print Server** → แตก zip ไว้ที่ `C:\` → ดับเบิลคลิก `install-autostart.bat` รายละเอียดทีละขั้นกดปุ่ม **คู่มือติดตั้ง** ข้างปุ่มดาวน์โหลด · อัปเดต = โหลด zip ใหม่แตกทับแล้วรัน `restart-print-server.bat`

หรือติดตั้งด้วย Git (อัปเดตด้วย `update-printer.bat`):

1. ติดตั้ง **Node.js LTS** และ **Git for Windows** (เหมือนข้อ 2.1)
2. PowerShell:

   ```
   cd C:\
   git config --global credential.useHttpPath true
   git clone https://github.com/Narai-IT/HumLai-pos.git humlai-pos
   ```

   ใช้ token อ่านอย่างเดียวแบบข้อ 3.1 (สร้างอีกตัวชื่อ `humlai-print-<สาขา>` ได้) · ไม่ใช้ Git → โหลด ZIP จาก GitHub แตกไฟล์ไว้ที่ `C:\humlai-pos`
3. ดับเบิลคลิก `C:\humlai-pos\install-autostart.bat` — ลงส่วนประกอบและตั้งให้ Print Server เปิดเองทุกครั้งที่ล็อกอินเครื่อง
4. เช็ก: เปิด `http://127.0.0.1:3001/health` ต้องมีข้อความตอบกลับ

ไม่ต้องมี .env บนเครื่อง Print Server
ที่อยู่ API ส่งมาจากหน้าหลังบ้าน (ข้อ 7.3) และเก็บไว้ใน `print-auto-config.json` ในเครื่องเอง

7.2 ตั้งปริ้นเตอร์ — หลังบ้าน → ปริ้นเตอร์ (เปิดจากเครื่อง Print Server)

1. ล็อกอินด้วยพนักงานของสาขานี้ → ต้องเห็น 🏠 **ปริ้นเตอร์ของสาขา**: ... ถูกสาขา
2. สถานะ Print Server ต้องเป็นสีเขียว (ถ้าแดงและหน้าเว็บเป็น https ให้กดใช้ `127.0.0.1` ในส่วนตั้งค่า Print Server)
3. กด **สแกนหาเครื่องพิมพ์** หรือเพิ่มเองด้วย IP → ตั้งชื่อ + ประเภท (ใบเสร็จ / ครัว / บาร์) → บันทึก
4. กดพิมพ์ทดสอบแต่ละเครื่อง

ตั้งชื่อปริ้นเตอร์ให้เหมือนกันทุกสาขา
เช่น "ครัว", "บาร์" — เมนูที่ตั้งให้ออกปริ้นเตอร์ "บาร์" ในเมนูกลาง จะไปออกปริ้นเตอร์ชื่อ "บาร์" ของแต่ละสาขาให้เอง

7.3 เปิดพิมพ์ใบครัวอัตโนมัติ (ออเดอร์ที่ลูกค้าสแกน QR สั่งเอง)

1. ในหน้าปริ้นเตอร์ กด **เปิดการพิมพ์อัตโนมัติ** (ระบบส่งที่อยู่ API + สาขา + หัวใบเสร็จให้ Print Server)
2. ถ้ามีกล่องสีเหลือง "Print Server ยังชี้ไปที่อยู่เดิม" → กด **อัปเดตให้ตรงกันเดี๋ยวนี้**
3. ทดสอบ: สแกน QR โต๊ะด้วยมือถือ สั่ง 1 รายการ → ภายใน 20 วินาทีใบครัวต้องออก

ผ่านแล้วเมื่อ
ขายที่หน้า POS 1 บิล ใบเสร็จออกพร้อมหัวใบของสาขา และออเดอร์จาก QR ออกใบครัวเอง

8. เปิดล็อก API

ขั้นสุดท้ายและสำคัญที่สุดด้านความปลอดภัย — หน้าพนักงานต้องล็อกอินเสมออยู่แล้ว แต่ก่อนเปิดล็อก API ใครรู้ที่อยู่ API ก็เรียกได้ทุกคำสั่งโดยไม่ผ่านหน้าเว็บ

1. เช็กว่าพนักงานทุกคนมีรหัส และมีแอดมินที่ตั้งรหัสแล้ว (ข้อ 6.2)
2. หลังบ้าน → ตั้งค่าร้าน → เปิด **ล็อก API: รับเฉพาะคำขอที่ล็อกอินแล้ว** → บันทึก
3. ออกจากระบบ แล้วลองล็อกอินด้วยรหัสพนักงานแต่ละคน
4. ลองสแกน QR สั่งอาหารอีกครั้ง (หน้าลูกค้าไม่ต้องล็อกอิน ต้องยังใช้ได้)

กลุ่ม	ทำได้
ไม่ต้องล็อกอิน	ลูกค้าสแกน QR สั่งและจ่าย, Print Server ดึงคิวใบครัว
พนักงานทั่วไป	ขาย จัดการโต๊ะ เช็กบิล — เฉพาะสาขาตัวเอง
แคชเชียร์	+ ผังโต๊ะ รับวัตถุดิบ ดูสต็อก
ผู้จัดการสาขา	+ ปริ้นเตอร์และเมนูรายสาขาของสาขาตัวเอง
แอดมินสำนักงานใหญ่	ทุกอย่าง: สาขา พนักงาน ตั้งค่าร้าน เมนูกลาง BOM ล้างข้อมูล

ใส่รหัสผิด 5 ครั้งติดกัน ผู้ใช้นั้นถูกล็อก 5 นาที · ล็อกอินหนึ่งครั้งใช้ได้ 12 ชั่วโมง · รายละเอียดใน `docs/LOGIN-AND-PERMISSIONS.md`

9. เปิดสาขาใหม่

ใช้เครื่องเซิร์ฟเวอร์ ฐานข้อมูล และหน้าเว็บเดิม — เพิ่มแค่ข้อมูลสาขาและ Print Server ของร้านใหม่

☐ หลังบ้าน → สาขา → เพิ่มสาขา (รหัส ชื่อ ตัวนำหน้าเลขบิล ที่อยู่ เบอร์ เลขภาษี) → บันทึก

☐ หลังบ้าน → พนักงาน → เพิ่มพนักงานของสาขาใหม่พร้อมรหัส (เลือกสาขาใหม่)

☐ ตั้งค่าร้าน → QR รับเงินของสาขาใหม่ (ถ้าบัญชีรับเงินต่างจากสาขาอื่น)

☐ ล็อกอินด้วยพนักงานสาขาใหม่ (หรือพนักงานทุกสาขาแล้วเลือกสาขาใหม่) → จัดการโต๊ะ → สร้างผังโต๊ะ → พิมพ์ QR

☐ เมนูรายสาขา → ปิดเมนูที่ไม่ขาย / ใส่ราคาที่ต่างจากเมนูกลาง

☐ ติดตั้ง Print Server ที่ร้านใหม่ (บทที่ 7) → ปริ้นเตอร์ชื่อเดียวกับสาขาอื่น → เปิดพิมพ์อัตโนมัติ

☐ สต็อก → เลือกสาขาใหม่ → รับวัตถุดิบเข้ายอดตั้งต้น (ถ้าใช้สต็อก)

☐ ทดลองขาย 1 บิล + สแกน QR 1 ครั้ง → เช็กใบเสร็จ ใบครัว และว่าไม่เห็นโต๊ะของสาขาอื่น

10. การอัปเดตระบบ

ส่วน	วิธีอัปเดต
หน้าเว็บ (Vercel)	อัตโนมัติทุกครั้งที่ merge เข้า `main` — ที่ร้านกด Ctrl+F5
API Server (เครื่องเซิร์ฟเวอร์)	Remote Desktop → คลิกขวา `D:\humlai-pos\update-api.bat` → Run as administrator ทำให้ครบ: ดึงโค้ด → ลงส่วนประกอบถ้าเปลี่ยน → `sql:init` → รีสตาร์ต API → ping ให้ดู
Print Server (แต่ละร้าน)	ดับเบิลคลิก `update-printer.bat` (ไม่ได้ใช้ Git → วาง ZIP ทับโฟลเดอร์แล้วรีสตาร์ตเครื่อง)

อัปเดตตอนร้านปิด
ถ้าขั้นไหนพัง สคริปต์หยุดก่อนรีสตาร์ต ตัวเดิมยังทำงานต่อ — แต่ควรทำนอกเวลาขายแล้วทดลองขาย 1 บิลก่อนเปิดร้าน

เช็กรุ่นที่ทำงานอยู่: `https://pos-api.<โดเมน>/api/pos?action=ping` → ค่า `build`

ห้าม `git reset --hard` ในโฟลเดอร์ที่ใช้งานอยู่
จะลบไฟล์ที่เคยแก้ไว้ในเครื่อง ถ้า `update-api.bat` บอกว่าไฟล์ในเครื่องถูกแก้ ให้ดูด้วย `git status` ก่อน

11. ความปลอดภัย

☐ เปิด **ล็อก API** (บทที่ 8)

☐ **ปิดพอร์ต 1433** ขาเข้าจากอินเทอร์เน็ต ทั้งที่เราเตอร์และ Windows Firewall — API ต่อฐานข้อมูลผ่าน `localhost` ไม่ต้องใช้พอร์ตนี้จากข้างนอก

☐ รหัสผ่าน `pos_app` ยาวและเดายาก เก็บแค่ใน `.env` ของเครื่องเซิร์ฟเวอร์ ไม่ใส่ในไฟล์ที่ขึ้น GitHub หรือส่งทางแชท

☐ ไม่เก็บค่า `SQL_*` ไว้ที่ Vercel

☐ คีย์ SlipOK อยู่ใน Environment Variables ของ Vercel เท่านั้น

☐ token GitHub ของเครื่องเซิร์ฟเวอร์/Print Server เป็นแบบ Read-only เฉพาะ repo — จดวันหมดอายุไว้ต่ออายุก่อนหมด

☐ สงสัยว่ามีคนได้ token ล็อกอินไป → ตั้ง `API_SECRET` ใหม่ใน `.env` แล้วรัน `update-api.bat` (ทุกเครื่องต้องล็อกอินใหม่)

12. แก้ปัญหาที่เจอบ่อย

อาการ	สาเหตุ / วิธีแก้
ping ขึ้น `Login failed for user 'pos_app'`	รหัสใน `.env` ผิด หรือมี `#` แต่ไม่ครอบด้วย `"..."` · ยังไม่ได้เปิด Mixed Mode
ping ขึ้น `Failed to connect`	SQL Server ไม่ทำงาน / ไม่ได้เปิด TCP/IP / ชื่ออินสแตนซ์ผิด (`localhost\SQLEXPRESS`)
ลิงก์ `pos-api...` ขึ้น error 502	Tunnel ทำงานแต่ API ดับ → ดู `D:\humlai-pos\logs\api-server.log` / รัน `update-api.bat`
ลิงก์ `pos-api...` ขึ้น error 1033 / Tunnel DOWN	cloudflared ไม่ทำงาน → PowerShell `sc.exe query cloudflared` / `sc.exe start cloudflared`
แถบแดง "บันทึกบิล ... ขึ้นระบบไม่สำเร็จ"	บิลไม่หาย ถูกเก็บในเครื่องและส่งซ้ำเองทุก 1.5 นาที อ่านสาเหตุในแถบ แล้วเช็ก ping · **ห้ามล้างข้อมูลเบราว์เซอร์**
หน้าเว็บยังเรียก API ที่อยู่เดิม	ยังไม่ได้ Redeploy หลังตั้ง `VITE_API_URL` / เบราว์เซอร์จำของเก่า → Ctrl+F5
หน้าเว็บขึ้น error เรื่อง CORS	`API_ALLOW_ORIGIN` ไม่ตรงกับโดเมนหน้าเว็บ (ต้องมี `https://` ไม่มี `/` ท้าย)
ใบเสร็จไม่ออก	Print Server ปิด (`http://127.0.0.1:3001/health`) / ปริ้นเตอร์ดับหรือกระดาษหมด / IP ปริ้นเตอร์เปลี่ยน
ออเดอร์ QR ไม่ออกใบครัว	ยังไม่เปิดพิมพ์อัตโนมัติ / มีกล่องเหลือง "ยังชี้ที่อยู่เดิม" → กดอัปเดต (ข้อ 7.3)
หัวใบเสร็จยังเป็นข้อความเดิม	Print Server รุ่นเก่า → `update-printer.bat` แล้วกดบันทึกการพิมพ์อัตโนมัติอีกครั้ง
ล็อกอินไม่ได้ "ใส่รหัสผิดหลายครั้ง"	รอ 5 นาที หรือให้แอดมินเช็กรหัสในหน้าพนักงาน
ล็อกอินแล้วเด้งกลับหน้าล็อกอินเรื่อย ๆ	API ยังเป็นรุ่นเก่า → `update-api.bat` · หรือเวลาเครื่องเพี้ยนมาก
เปิดล็อก API ไม่ได้	ต้องมีแอดมินที่ตั้งรหัสไว้อย่างน้อย 1 คนก่อน
ไม่เห็นโต๊ะ/บิล/เมนูที่ควรเห็น	ล็อกอินผิดสาขา — ดูชื่อสาขามุมบนหน้าขาย · พนักงาน "ทุกสาขา" ออกจากระบบแล้วเลือกสาขาใหม่
`update-api.bat` : ดึงโค้ดไม่สำเร็จ	token GitHub หมดอายุ → Credential Manager → ลบเฉพาะ `git:https://github.com/Narai-IT/HumLai-pos.git` แล้วรันใหม่ใส่ token ใหม่

คู่มือเพิ่มเติมใน GitHub (โฟลเดอร์ `docs/`)

ไฟล์	เรื่อง
`CLOUDFLARE-TUNNEL.md`	Tunnel แบบละเอียด / ใช้ Tunnel ที่มีอยู่แล้ว / Quick Tunnel
`UPDATE-API-WITH-GIT.md`	เชื่อมโฟลเดอร์เดิมกับ Git และ update-api.bat
`PRINT-SERVER-AFTER-API-MOVE.md`	Print Server, เช็กทุกเช้า, หลายสาขา

`LOGIN-AND-PERMISSIONS.md`	ล็อกอินและสิทธิ์
`SQL-MIGRATION.md`	ย้ายข้อมูลจาก Google Sheet